

Apache Spark

Architecture, RDDs, DataFrames & Docker Setup

1. What is Apache Spark?

Apache Spark is an open-source, distributed computing system used for processing and analyzing large datasets. Unlike Hadoop MapReduce, which writes intermediate results to disk, Spark processes data primarily in-memory (RAM), making it up to 100x faster for certain workloads. It is the backbone of modern big data and data engineering pipelines.

2. Spark Architecture

Spark follows a **Master-Slave (Master-Worker)** architecture. When you submit a Spark application, it breaks the work down and distributes it across a cluster of machines.

- **Driver Program:** The "brain" of the application. It runs your `main()` function, creates the `SparkContext` (or `SparkSession`), and decides how to split the work into smaller tasks.
- **Cluster Manager:** The resource manager. It allocates physical resources (CPU, RAM) across the cluster. Examples include Spark Standalone, YARN, Mesos, and Kubernetes.
- **Worker Nodes:** The "muscle." These are the machines that actually run the computations. Each Worker has **Executors** (processes that run the code and store data in cache or memory) and executes **Tasks** (the smallest unit of work).

3. RDDs (Resilient Distributed Datasets)

The Foundation of Spark

RDD is the fundamental data structure of Apache Spark. Think of it as a low-level, immutable collection of objects distributed across the cluster.

- **Resilient:** Fault-tolerant. If a node crashes, Spark can automatically rebuild the lost partition of the RDD using the "lineage" (the history of operations used to build it).
- **Distributed:** Data is split into partitions and sent to different worker nodes.

- **Lazy Evaluation:** Spark doesn't actually compute anything when you define an RDD transformation (like ``map`` or ``filter``). It waits until an **Action** (like ``count`` or ``collect``) is called. This allows Spark to optimize the execution plan.

Note: In modern Spark, you rarely write raw RDD code unless you are dealing with deeply unstructured data or need low-level API control.

4. DataFrames

Introduced in Spark 1.3, DataFrames sit on top of RDDs but organize data into named columns, exactly like a table in a relational database or a Pandas DataFrame.

- **Higher-Level Abstraction:** You can use SQL queries or DataFrame API methods (``df.select()``, ``df.groupBy()``) instead of writing complex Python lambda functions.
- **Catalyst Optimizer:** This is the secret weapon of DataFrames. Spark analyzes your query, optimizes it (e.g., filtering data **before** joining tables), and generates highly efficient Java bytecode to run on the JVM.
- **Schema:** DataFrames have strict schemas (data types for columns), making it easier to handle structured and semi-structured data (CSV, JSON, Parquet).

5. Installation & Setup (Using Docker)

Since you are already familiar with Docker, the cleanest way to set up a Spark playground without polluting your local machine is using a ``docker-compose.yml`` file. This setup will create a standalone cluster with one Master and one Worker.

Step 1: Create ``docker-compose.yml``

```
version: '3.8'

services:
  spark-master:
    image: bitnami/spark:3.5
    environment:
      - SPARK_MODE=master
    ports:
      - '8080:8080' # Spark Master Web UI
      - '7077:7077' # Master port for workers to connect
    volumes:
      - ./data:/opt/bitnami/spark/data

  spark-worker:
    image: bitnami/spark:3.5
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_MEMORY=1G
      - SPARK_WORKER_CORES=1
    depends_on:
      - spark-master
    volumes:
      - ./data:/opt/bitnami/spark/data
```

Step 2: Start the Cluster

Open your terminal in the same folder and run:

```
docker-compose up -d
```

You can view the Spark Master Web UI by going to <http://localhost:8080> in your browser. You should see one worker registered and alive.

Step 3: Run a PySpark Job

You can execute PySpark interactively by opening a shell inside your master container:

```
docker exec -it <your_master_container_name> pyspark --master spark://spark-master:7077
```

Once inside the PySpark shell, you can start testing DataFrame operations directly!